UNIVERSIDAD TECNOLÓGICA DE GUTIÉRREZ ZAMORA

INGENIERÍA EN DESARROLLO Y GESTIÓN DE SOFTWARE

REPORTE DE APLICACIÓN

DESARROLLO MÓVIL INTEGRAL

POR:

JOSÉ ANTONIO HERNÁNDEZ CRUZ

ALONDRA SAN MARTÍN HERNANDEZ

GLORIA ANGÉLICA DÍAZ OLARTE

DOCENTE:

YOSELYN NOHEMI ORTEGA GIJON

GRADO: 1010-AEI

SEPTIEMBRE – DICIEMBRE 2026

Contenido

PLAN DEL PROCESO DE DESARROLLO MÓVIL SEGÚN LA METODOLOGÍA SCRUM 4

ROLES DE SCRUM 5

ARTEFACTOS DE SCRUM 6

EVENTOS DE SCRUM 7

PLANIFICACIÓN DE LOS SPRINTS 8

JUSTIFICACIÓN DE LA ARQUITECTURA MVVM 10

SEPARACIÓN DE RESPONSABILIDADES 10

MANTENIBILIDAD Y ORGANIZACIÓN 11

COMUNICACIÓN CON LOS SERVICIOS DEL SISTEMA 11

FACILIDAD PARA REALIZAR PRUEBAS 12

DIAGRAMA DE LA ARQUITECTURA 13

PROPUESTA DE LOS PATRONES DE DISEÑO 14

PATRÓN SINGLETON 14

PATRÓN OBSERVER 15

JUSTIFICACIÓN DE LOS FRAMEWORKS DE DESARROLLO MÓVIL A UTILIZAR 17

IONIC 17

ANGULAR 18

INTEGRACIÓN DE IONIC Y ANGULAR 19

ESQUEMA DE PRUEBAS 20

PRUEBAS UNITARIAS 20

PRUEBAS DE INTEGRACIÓN 21

PRUEBAS END-TO-END (E2E) 21

PRUEBAS MANUALES EN DISPOSITIVOS MÓVILES 22

ESTRATEGIA DE VERSIONAMIENTO 23

FLUJO DE RAMAS 23

VERSIONADO SEMÁNTICO 24

REGISTRO DE CAMBIOS 25

PLAN DEL PROCESO DE DESARROLLO MÓVIL SEGÚN LA METODOLOGÍA SCRUM

Para el desarrollo de la Plataforma Integral para la Gestión y Seguimiento de Proyectos en BEPARTNERS se seleccionó la metodología ágil Scrum, debido a que permite organizar el proceso de desarrollo de manera incremental mediante ciclos de trabajo denominados Sprints. Este enfoque facilita la distribución de las funcionalidades del sistema en entregas parciales, permitiendo revisar los avances obtenidos y realizar ajustes conforme evoluciona el proyecto.

La implementación de esta metodología resulta adecuada para el desarrollo de la aplicación móvil, debido a que permitirá incorporar de manera progresiva las funcionalidades destinadas a los integrantes de los equipos de trabajo de BEPARTNERS. Entre estas se encuentran la consulta de tareas asignadas, visualización de fechas de compromiso, actualización del progreso, registro de actividades completadas, comentarios de seguimiento, consulta de acuerdos, notificaciones y recordatorios. De esta manera, cada funcionalidad podrá desarrollarse y validarse de forma incremental antes de integrarse a la versión final de la aplicación.

El proceso de desarrollo estará organizado en cinco Sprints con una duración de dos semanas cada uno, estableciendo objetivos específicos para cada periodo. Al finalizar cada Sprint se obtendrá un incremento funcional del sistema, el cual será revisado antes de continuar con la siguiente etapa de desarrollo.

ROLES DE SCRUM

Para llevar a cabo el proceso de desarrollo se establecerán los principales roles definidos por Scrum, asignando responsabilidades específicas durante la ejecución del proyecto.

- **Product Owner:** será responsable de representar las necesidades del proyecto y establecer la prioridad de los requerimientos que conforman el Product Backlog. Asimismo, verificará que las funcionalidades desarrolladas correspondan con las necesidades identificadas para la gestión y seguimiento de proyectos.
- **Scrum Master:** será responsable de facilitar la aplicación de la metodología, coordinar los eventos de Scrum y apoyar en la identificación de obstáculos que puedan afectar el cumplimiento de los objetivos establecidos para cada Sprint.
- **Equipo de desarrollo:** estará integrado por los responsables del análisis, diseño, programación, integración y pruebas de la aplicación móvil. Sus integrantes desarrollarán las funcionalidades definidas para cada Sprint, manteniendo la comunicación y coordinación necesarias para garantizar la correcta integración de los diferentes componentes de la aplicación.

ARTEFACTOS DE SCRUM

Durante el desarrollo se utilizarán los artefactos de Scrum para organizar y controlar las actividades necesarias para la construcción del sistema.

- **Product Backlog:** concentrará los requerimientos y funcionalidades correspondientes a la aplicación móvil. Incluirá el acceso de los usuarios, consulta de tareas asignadas, visualización de fechas de compromiso, actualización del progreso de las actividades, registro de actividades completadas, comentarios de seguimiento, consulta de acuerdos, notificaciones, recordatorios y consulta de actividades pendientes o atrasadas.
- **Sprint Backlog:** contendrá las actividades seleccionadas del Product Backlog que deberán desarrollarse durante cada Sprint. Su contenido estará determinado por el objetivo establecido para cada periodo de trabajo.
- **Incremento del producto:** estará conformado por las funcionalidades terminadas y verificadas al concluir cada Sprint. Los incrementos serán integrados progresivamente hasta conformar la versión completa de la aplicación móvil.

EVENTOS DE SCRUM

El proceso de desarrollo de cada Sprint iniciará con el **Sprint Planning**, donde se establecerá el objetivo del periodo y se seleccionarán las actividades del Product Backlog que formarán parte del Sprint Backlog.

Durante el desarrollo se realizarán reuniones breves de seguimiento mediante el **Daily Scrum**, con la finalidad de identificar el avance de las actividades, coordinar el trabajo del equipo y detectar posibles impedimentos.

Al concluir cada periodo se realizará el **Sprint Review**, en el cual se presentará y verificará el incremento desarrollado, comprobando el cumplimiento de las funcionalidades establecidas para el Sprint.

Finalmente, mediante el **Sprint Retrospective**, se analizará el desempeño del equipo y el proceso seguido durante el Sprint, identificando aspectos que puedan mejorarse para los siguientes ciclos de desarrollo.

PLANIFICACIÓN DE LOS SPRINTS

El proceso de desarrollo de la aplicación móvil se organizará en cinco Sprints de dos semanas. Cada Sprint estará enfocado en la implementación progresiva de las funcionalidades destinadas a los integrantes de los equipos de trabajo de BEPARTNERS, desde la definición inicial de la aplicación hasta su integración y validación final.

Sprint	Objetivo	Actividades principales
Sprint 1. Análisis y diseño de la aplicación móvil	Definir los requerimientos, estructura y diseño de la aplicación.	Análisis de requerimientos móviles, definición de los flujos de navegación, identificación de los datos necesarios, elaboración de interfaces y mockups y preparación de la estructura inicial del proyecto móvil.
Sprint 2. Acceso y consulta de actividades	Implementar el acceso del usuario y las funciones principales de consulta.	Autenticación del usuario, consulta de tareas asignadas, visualización de fechas de compromiso y consulta de actividades pendientes y atrasadas.
Sprint 3. Seguimiento y actualización de actividades	Permitir que los integrantes del equipo gestionen el avance de sus actividades desde la aplicación.	Visualización del detalle de las actividades, actualización del progreso, registro de actividades completadas y consulta de acuerdos relacionados.
Sprint 4. Comentarios,	Incorporar las funciones de comunicación y	Registro de comentarios e información de seguimiento,

notificaciones y recordatorios	seguimiento oportuno de las actividades.	recepción de notificaciones sobre nuevas tareas o cambios y recordatorios de actividades próximas a vencer.
Sprint 5. Integración, pruebas y ajustes	Validar el funcionamiento completo de la aplicación móvil y su comunicación con los servicios del sistema.	Integración de funcionalidades, validación de la comunicación con el backend, pruebas de autenticación y permisos, pruebas funcionales, corrección de errores y ajustes finales de interfaz y usabilidad.

De esta manera, Scrum permitirá mantener un proceso de desarrollo móvil organizado e incremental, en el que cada Sprint contribuirá a la incorporación progresiva de nuevas funcionalidades hasta obtener una versión integrada y funcional de la aplicación móvil. La revisión continua de los incrementos permitirá identificar errores y realizar los ajustes necesarios durante el desarrollo, evitando concentrar la validación del producto únicamente en la etapa final.

JUSTIFICACIÓN DE LA ARQUITECTURA MVVM

Para el desarrollo de la aplicación móvil de la Plataforma Integral para la Gestión y Seguimiento de Proyectos en BEPARTNERS se utilizará la arquitectura MVVM (Model–View–ViewModel). Esta arquitectura permite organizar la aplicación mediante la separación de la interfaz de usuario, la lógica encargada de gestionar su estado y los datos utilizados por el sistema.

La aplicación móvil requiere consultar y actualizar constantemente información relacionada con las actividades de los usuarios, como tareas asignadas, fechas de compromiso, progreso, comentarios y acuerdos. Asimismo, deberá recibir notificaciones relacionadas con nuevas tareas, cambios y actividades próximas a vencer. Debido a estas características, MVVM permitirá mantener organizada la lógica necesaria para presentar y actualizar esta información sin concentrarla directamente en las interfaces de la aplicación.

SEPARACIÓN DE RESPONSABILIDADES

La arquitectura MVVM divide la aplicación en tres componentes principales:

Model: representa los datos y la lógica relacionada con la información utilizada por la aplicación. En este proyecto incluirá elementos como usuarios, proyectos, actividades, acuerdos, comentarios y notificaciones. También permitirá gestionar la información obtenida o enviada mediante los servicios correspondientes.

View: corresponde a las interfaces con las que interactuará el usuario. Incluye las pantallas de inicio de sesión, tareas asignadas, detalle de actividades, acuerdos, comentarios, pendientes y demás elementos visuales de la aplicación. Su función principal será mostrar la información y capturar las acciones realizadas por el usuario.

ViewModel: actuará como intermediario entre la View y los datos utilizados por la aplicación. Será responsable de gestionar el estado de las pantallas y preparar la información que deberá mostrarse al usuario. También procesará las acciones realizadas desde la interfaz, como actualizar el progreso de una actividad, marcarla como completada o registrar un comentario.

MANTENIBILIDAD Y ORGANIZACIÓN

La separación de responsabilidades proporcionada por MVVM facilitará el mantenimiento de la aplicación, ya que permitirá modificar las interfaces sin afectar directamente la lógica encargada de gestionar los datos. De la misma manera, los cambios realizados en la forma de obtener o procesar información podrán implementarse sin modificar innecesariamente las pantallas de la aplicación.

Esta organización también evitará concentrar grandes cantidades de lógica dentro de los componentes visuales, permitiendo mantener una estructura de código más clara y facilitar la incorporación de nuevas funcionalidades durante los diferentes Sprints del proyecto.

COMUNICACIÓN CON LOS SERVICIOS DEL SISTEMA

La aplicación móvil formará parte de una plataforma en la que la información de los proyectos se encontrará centralizada y será compartida con la aplicación web. Para ello, la aplicación móvil se comunicará con los servicios del sistema mediante una API, encargada de proporcionar y recibir la información necesaria.

Dentro de la arquitectura MVVM, la comunicación con la API se realizará mediante servicios de Angular utilizando HttpClient, evitando que las páginas y componentes de Ionic gestionen directamente las solicitudes al servidor. Los datos obtenidos serán representados mediante modelos o interfaces y procesados antes de mostrarse en las diferentes vistas. Esta separación permitirá mantener organizada

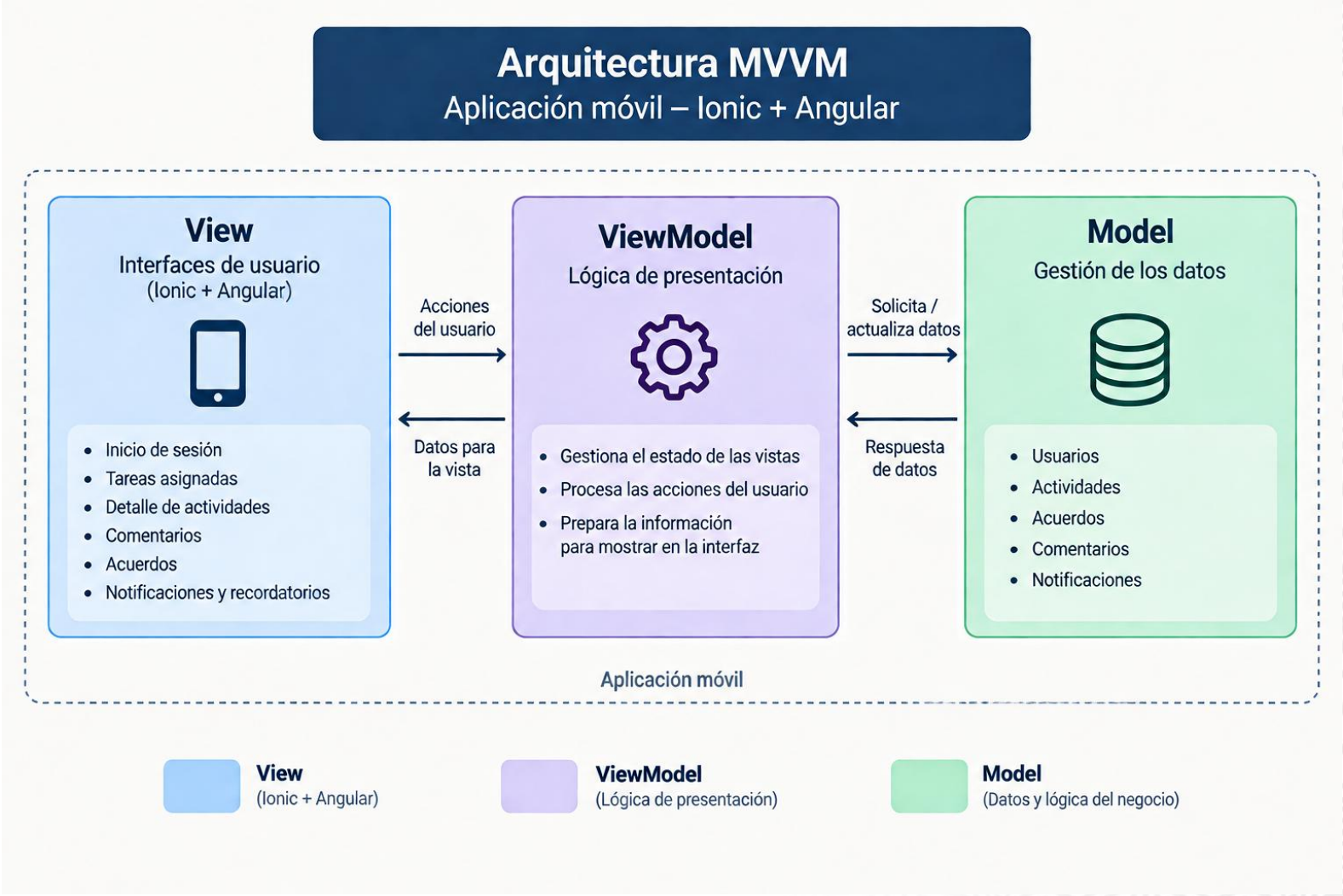
la comunicación con el backend y facilitará operaciones como la consulta de tareas, actualización del progreso de actividades y registro de comentarios.

FACILIDAD PARA REALIZAR PRUEBAS

La separación entre la interfaz y la lógica de presentación facilitará la realización de pruebas sobre las funcionalidades de la aplicación. La lógica contenida en los ViewModels podrá validarse independientemente de las pantallas, permitiendo comprobar operaciones como la actualización del progreso, el manejo de estados o la preparación de información antes de mostrarla al usuario.

Por estas razones, la arquitectura MVVM proporciona una estructura adecuada para organizar el desarrollo de la aplicación móvil de BEPARTNERS, manteniendo separadas las responsabilidades de presentación, gestión del estado y manejo de datos, lo que favorecerá su mantenimiento, crecimiento y validación durante el proceso de desarrollo.

DIAGRAMA DE LA ARQUITECTURA



PROPUESTA DE LOS PATRONES DE DISEÑO

Para el desarrollo de la aplicación móvil de BEPARTNERS se utilizarán los patrones de diseño Singleton y Observer, con el propósito de organizar las responsabilidades del código y facilitar el mantenimiento de las diferentes funcionalidades de la aplicación. Su implementación permitirá administrar servicios compartidos y mantener actualizada la información presentada al usuario durante la interacción con el sistema.

PATRÓN SINGLETON

Clasificación: Patrón creacional.

Justificación:

La aplicación móvil requiere utilizar servicios compartidos desde diferentes pantallas, principalmente para gestionar la autenticación, mantener la información de la sesión y realizar operaciones comunes durante la navegación del usuario.

El patrón Singleton permitirá mantener una única instancia de estos servicios durante la ejecución de la aplicación. Esto evitará la creación innecesaria de múltiples instancias y permitirá que los diferentes componentes trabajen con la misma información mientras el usuario utiliza las funciones disponibles.

Implementación en el proyecto:

El patrón Singleton se implementará mediante los servicios de Angular utilizados en la aplicación. Principalmente, se aplicará en el servicio encargado de la autenticación, el cual mantendrá disponible la información necesaria de la sesión del usuario mientras este navega entre las diferentes pantallas.

De esta manera, las vistas correspondientes a tareas asignadas, actividades, acuerdos y notificaciones podrán utilizar el mismo servicio de autenticación sin generar una instancia independiente para cada componente.

Beneficios:

- **Consistencia de la sesión:** permitirá mantener disponible la información del usuario durante la navegación entre las diferentes pantallas de la aplicación.
- **Uso eficiente de recursos:** evitará la creación innecesaria de múltiples instancias de un mismo servicio.
- **Acceso compartido:** permitirá que diferentes componentes utilicen una misma instancia para acceder a funcionalidades comunes.
- **Organización del código:** facilitará la centralización de operaciones que deben estar disponibles en distintas partes de la aplicación.

PATRÓN OBSERVER

Clasificación: Patrón de comportamiento.

Justificación:

La aplicación móvil requiere que diferentes elementos de la interfaz respondan a los cambios producidos en la información durante su utilización. Por ejemplo, cuando un usuario actualice el progreso de una actividad o la marque como completada, las vistas relacionadas deberán reflejar el nuevo estado de la información.

Este comportamiento resulta adecuado para las funciones establecidas en la aplicación móvil de BEPARTNERS, debido a que los integrantes de los equipos podrán consultar sus tareas, actualizar el progreso de las actividades, marcarlas como completadas y recibir información relacionada con nuevas tareas o modificaciones.

Implementación en el proyecto:

El patrón Observer se implementará mediante los mecanismos reactivos utilizados por Angular y RxJS. Los componentes interesados se suscribirán a los cambios gestionados por los servicios de la aplicación, permitiendo actualizar la información presentada cuando se produzcan modificaciones en los datos.

Por ejemplo, al actualizar el progreso de una actividad, el servicio encargado de gestionar las actividades notificará el cambio a los componentes suscritos. De esta manera, las vistas relacionadas podrán mostrar el nuevo estado sin necesidad de gestionar manualmente la actualización desde cada componente.

Beneficios:

- **Actualización de la interfaz:** permitirá reflejar los cambios realizados en la información utilizada por las diferentes vistas.
- **Reducción del acoplamiento:** los componentes podrán reaccionar a los cambios sin depender directamente unos de otros.
- **Comunicación entre componentes y servicios:** facilitará la distribución de cambios de estado dentro de la aplicación.
- **Manejo reactivo de la información:** permitirá gestionar de manera organizada los cambios producidos durante la interacción del usuario con las actividades del sistema.

JUSTIFICACIÓN DE LOS FRAMEWORKS DE DESARROLLO MÓVIL A UTILIZAR

Para el desarrollo de la aplicación móvil de BEPARTNERS se utilizarán los frameworks Ionic y Angular, debido a que proporcionan las herramientas necesarias para construir una aplicación multiplataforma, organizar sus diferentes funcionalidades y mantener una estructura de desarrollo modular.

La aplicación estará dirigida principalmente a los integrantes de los equipos de trabajo, permitiéndoles consultar tareas asignadas, visualizar fechas de compromiso, actualizar el progreso de sus actividades, registrar comentarios, consultar acuerdos y recibir notificaciones y recordatorios. La utilización conjunta de Ionic y Angular permitirá desarrollar estas funcionalidades manteniendo una misma estructura tecnológica y facilitando su integración con los servicios encargados de proporcionar la información del sistema.

IONIC

Ionic se utilizará como framework principal para el desarrollo de la interfaz de la aplicación móvil. Su estructura basada en componentes permitirá construir las diferentes pantallas requeridas por el sistema y mantener una presentación uniforme durante la navegación del usuario.

El framework permitirá desarrollar interfaces para la consulta de tareas, visualización del detalle de las actividades, actualización del progreso, registro de comentarios, consulta de acuerdos y visualización de notificaciones. La reutilización de componentes facilitará mantener una estructura consistente entre estas funcionalidades y reducirá la duplicación de elementos durante el desarrollo.

Asimismo, Ionic permite desarrollar aplicaciones mediante tecnologías web como HTML, CSS y TypeScript, manteniendo una misma base de código para diferentes plataformas móviles. Esto facilitará el mantenimiento de la aplicación al evitar la

necesidad de desarrollar de manera independiente las mismas funcionalidades para cada sistema operativo.

Para la integración con las funcionalidades nativas del dispositivo se utilizará Capacitor, permitiendo que la aplicación pueda incorporar capacidades propias de los dispositivos móviles cuando sean requeridas. Su integración también permitirá conservar la estructura desarrollada con Ionic y Angular sin separar la aplicación en implementaciones independientes.

ANGULAR

Angular se utilizará para organizar la lógica y el funcionamiento de la aplicación móvil. Su estructura basada en componentes y servicios permitirá distribuir las responsabilidades de las diferentes funcionalidades y mantener el código organizado conforme avance el desarrollo.

El uso de TypeScript permitirá definir de manera estructurada los datos utilizados por la aplicación, incluyendo usuarios, actividades, acuerdos, comentarios y notificaciones. El tipado de estos elementos contribuirá a mantener consistencia en el manejo de la información y facilitará la identificación de errores durante el desarrollo.

La inyección de dependencias proporcionada por Angular permitirá utilizar servicios compartidos desde diferentes componentes de la aplicación. Esta característica será utilizada principalmente para centralizar funciones como la autenticación, gestión de actividades y comunicación con los servicios del sistema, manteniendo separadas estas responsabilidades de las interfaces de usuario.

Asimismo, los mecanismos reactivos utilizados junto con Angular permitirán implementar el patrón Observer definido para el proyecto. Los componentes podrán responder a cambios en la información y actualizar su contenido cuando se modifique el estado de las actividades u otros datos utilizados por la aplicación.

INTEGRACIÓN DE IONIC Y ANGULAR

La integración de Ionic y Angular permitirá combinar los componentes necesarios para construir la interfaz móvil con una estructura organizada para administrar la lógica de la aplicación. Esta separación contribuirá a la implementación de la arquitectura MVVM, manteniendo diferenciadas las responsabilidades relacionadas con la presentación, gestión del estado y manejo de los datos.

La aplicación móvil deberá intercambiar información con los servicios centrales de la plataforma, debido a que el sistema de BEPARTNERS concentrará la información relacionada con proyectos, actividades, acuerdos y pendientes en una plataforma común. Para mantener organizada esta comunicación, los servicios desarrollados con Angular serán responsables de realizar las operaciones necesarias para consultar y actualizar la información utilizada por la aplicación.

La utilización conjunta de Ionic y Angular proporcionará una estructura adecuada para el desarrollo de la aplicación móvil de BEPARTNERS, permitiendo mantener una base de código organizada, reutilizar componentes y facilitar la incorporación progresiva de las funcionalidades establecidas durante los diferentes Sprints del proyecto.

ESQUEMA DE PRUEBAS

Para garantizar el correcto funcionamiento de la aplicación móvil de BEPARTNERS se utilizará un esquema basado en la Pirámide de Pruebas, permitiendo verificar el sistema en diferentes niveles durante el proceso de desarrollo. El esquema estará compuesto por pruebas unitarias, pruebas de integración, pruebas End-to-End (E2E) y pruebas manuales en dispositivos móviles.

La aplicación permitirá realizar operaciones relacionadas con la consulta y seguimiento de actividades, incluyendo la visualización de tareas asignadas y fechas de compromiso, actualización del progreso, registro de actividades completadas, comentarios, consulta de acuerdos y recepción de notificaciones y recordatorios. Estas funcionalidades serán verificadas progresivamente conforme sean incorporadas durante los diferentes Sprints.

PRUEBAS UNITARIAS

Las pruebas unitarias se utilizarán para comprobar de manera independiente el funcionamiento de componentes, servicios y funciones específicas de la aplicación.

Estas pruebas permitirán verificar la lógica relacionada con el manejo de actividades, cambios de estado, validación de información y procesamiento de los datos utilizados por las diferentes vistas. También permitirán comprobar el comportamiento de los servicios desarrollados con Angular sin depender de la ejecución completa de la aplicación.

Su aplicación facilitará la detección temprana de errores durante el desarrollo y permitirá realizar modificaciones en el código verificando que las funcionalidades previamente desarrolladas continúen funcionando correctamente.

PRUEBAS DE INTEGRACIÓN

Las pruebas de integración verificarán la comunicación entre los diferentes componentes y servicios de la aplicación, así como el intercambio de información con los servicios centrales de la plataforma.

Se comprobarán operaciones como la autenticación del usuario, consulta de tareas asignadas, actualización del progreso de una actividad, registro de comentarios y obtención de acuerdos. También se validará que la información enviada desde la aplicación sea procesada correctamente y que los datos recibidos puedan mostrarse de manera adecuada en las diferentes vistas.

Estas pruebas tendrán especial importancia debido a que la aplicación móvil compartirá información con la plataforma central utilizada para gestionar los proyectos de BEPARTNERS.

PRUEBAS END-TO-END (E2E)

Las pruebas End-to-End permitirán validar flujos completos de utilización de la aplicación desde la perspectiva del usuario. Su objetivo será comprobar que las diferentes funciones trabajen correctamente de manera conjunta desde el inicio hasta la finalización de una operación.

Entre los principales flujos que serán verificados se encontrarán:

- Inicio de sesión → consulta de tareas asignadas → selección de una actividad → actualización de su progreso.
- Consulta de actividad → registro de comentario → visualización de la información actualizada.
- Consulta de tareas pendientes → selección de actividad → actualización como completada.
- Recepción de una notificación → acceso a la aplicación → consulta de la actividad relacionada.

La ejecución de estos escenarios permitirá comprobar que la navegación, la lógica de la aplicación y el manejo de la información funcionen correctamente de manera integrada.

PRUEBAS MANUALES EN DISPOSITIVOS MÓVILES

Finalmente, se realizarán pruebas manuales en dispositivos móviles para verificar el comportamiento de la aplicación en condiciones reales de uso. Estas pruebas permitirán evaluar aspectos relacionados con la navegación, adaptación de las interfaces, legibilidad de la información, interacción mediante pantalla táctil y comportamiento general de la aplicación.

También se verificará el funcionamiento de las notificaciones y recordatorios establecidos para las actividades, así como el comportamiento de la aplicación ante diferentes condiciones de conectividad.

La combinación de estos niveles de prueba permitirá detectar errores tanto en funciones individuales como en la integración y utilización completa de la aplicación, contribuyendo a mantener la calidad de cada incremento generado durante el proceso de desarrollo.

ESTRATEGIA DE VERSIONAMIENTO

Para administrar los cambios realizados durante el desarrollo de la aplicación móvil de BEPARTNERS se utilizará Git como sistema de control de versiones y GitHub como plataforma para almacenar y gestionar el repositorio del proyecto.

La estrategia estará basada en un flujo de ramas inspirado en Git Flow, complementado con Versionado Semántico (Semantic Versioning). Esta organización permitirá separar el código estable del código que se encuentre en desarrollo y mantener un registro de las modificaciones realizadas durante los diferentes Sprints.

FLUJO DE RAMAS

El repositorio se organizará mediante diferentes ramas de acuerdo con el estado y propósito de los cambios realizados.

main: contendrá únicamente las versiones estables y verificadas de la aplicación. Los cambios serán incorporados a esta rama una vez que hayan sido integrados y probados correctamente.

develop: funcionará como la rama principal de desarrollo e integración. En ella se concentrarán las funcionalidades terminadas antes de incorporarlas a una versión estable.

feature/*: se utilizarán ramas independientes para desarrollar nuevas funcionalidades. Cada funcionalidad será trabajada sin modificar directamente las ramas principales y posteriormente será integrada a develop.

Por ejemplo:

feature/login

feature/tareas

feature/comentarios

feature/notificaciones

fix/*: se utilizarán para realizar correcciones de errores identificados durante el desarrollo y las pruebas antes de generar una versión estable

Esta estructura permitirá que los integrantes del equipo trabajen en diferentes funcionalidades sin realizar modificaciones directamente sobre el código estable de la aplicación.

VERSIONADO SEMÁNTICO

Las versiones estables de la aplicación utilizarán el formato:

vX.Y.Z

donde cada número representará el tipo de cambio realizado.

X – Versión mayor: se incrementará cuando se realicen cambios importantes que modifiquen significativamente el funcionamiento de la aplicación o generen incompatibilidades con versiones anteriores.

Y – Versión menor: se incrementará cuando se incorporen nuevas funcionalidades manteniendo compatibilidad con las características existentes.

Z – Parche: se incrementará cuando se realicen correcciones de errores o ajustes menores que no modifiquen las funcionalidades principales.

Por ejemplo, una versión identificada como v1.2.1 representará la versión mayor 1, con dos actualizaciones de funcionalidades y una corrección correspondiente a esa versión.

REGISTRO DE CAMBIOS

Los cambios realizados durante el desarrollo se registrarán mediante commits descriptivos, permitiendo identificar con claridad la finalidad de cada modificación.

Se utilizarán prefijos que permitan clasificar los principales tipos de cambios, por ejemplo:

feat: incorporación de una nueva funcionalidad.

fix: corrección de un error.

docs: modificación de documentación.

test: incorporación o modificación de pruebas.

refactor: reorganización del código sin modificar su comportamiento.

De esta manera, la estrategia de versionamiento permitirá mantener un historial organizado del desarrollo, facilitar el trabajo colaborativo y relacionar las funcionalidades implementadas durante los Sprints con las diferentes versiones de la aplicación móvil.